Décomposez les tâches complexes en requêtes distinctes

Pour les tâches complexes qui nécessitent plusieurs instructions ou étapes, vous pouvez améliorer les réponses du modèle en divisant vos requêtes en sous-tâches. Des requêtes plus petites peuvent vous aider à améliorer la contrôlabilité, le débogage et la précision.

Il existe deux façons de décomposer les requêtes complexes et de les ingérer dans un modèle :

- **Requêtes de chaîne** : divisez une tâche en sous-tâches et exécutez les sous-tâches de manière séquentielle.
- **Réponses agrégées**: divisez une tâche en sous-tâches et exécutez les sous-tâches en parallèle.

Requêtes de chaîne

Pour les tâches complexes impliquant plusieurs étapes *séquentielles*, faites de chaque étape une requête et associez les requêtes dans une séquence. Dans cette chaîne de requêtes séquentielle, la sortie d'une requête de la séquence devient l'entrée de la requête suivante. Le résultat de la dernière requête de la séquence est le résultat final.

Exemple

Par exemple, supposons que vous dirigez une entreprise de télécommunications et que vous souhaitez utiliser un modèle pour vous aider à analyser les commentaires des clients afin d'identifier les problèmes courants des clients, de classer les problèmes en catégories et de générer des solutions pour des catégories de problèmes.

Tâche 1 : identifier les problèmes des clients

La première tâche que vous souhaitez que le modèle effectue consiste à extraire des données pertinentes à partir des commentaires bruts des clients. Une requête permettant d'effectuer cette tâche peut être semblable à la suivante, où *`CUSTOMER_FEEDBACK`* est un fichier contenant les commentaires des clients :

---

e les données

---

e :

```
act the main issues and sentiments from the customer feedback on our telecom servic
s on comments related to service disruptions, billing issues, and customer support
actions.
se format the output into a list with each issue/sentiment in a sentence, separated
colon.

: CUSTOMER_FEEDBACK
```

La réponse du modèle devrait contenir une liste de problèmes et de sentiments extraits des commentaires des clients.

Tâche 2 : classer les problèmes en catégories

Vous souhaitez ensuite indiquer au modèle de classer les données en catégories afin de pouvoir comprendre les types de problèmes auxquels les clients sont confrontés, en utilisant la réponse de la tâche précédente. Une requête permettant d'effectuer cette tâche peut ressembler à la suivante, où *TASK_1_RESPONSE* correspond à la réponse de la tâche précédente :

er les données

```
e :

sify the extracted issues into categories such as service reliability, pricing
erns, customer support quality, and others.
se organize the output into JSON format with each issue as the key, and category as
e.

: TASK_1_RESPONSE
```

La réponse du modèle devrait contenir des problèmes catégorisés.

Tâche 3 : générer des solutions

Vous allez maintenant demander au modèle de générer des recommandations exploitables en fonction des problèmes catégorisés afin d'améliorer la satisfaction des clients, en utilisant la réponse de la tâche précédente. Une requête qui effectue cette opération peut ressembler à la suivante, où *TASK_2_RESPONSE* est la réponse de la tâche précédente :

er des suggestions

```
e:

rate detailed recommendations for each category of issues identified from the feedb
est specific actions to address service reliability, improving customer support, an
sting pricing models, if necessary.
se organize the output into a JSON format with each category as the key, and
mendation as the value.

:: TASK_2_RESPONSE
```

La réponse du modèle devrait contenir des recommandations pour chaque catégorie, visant à améliorer l'expérience client et la qualité du service, ce qui correspond à notre objectif général.

Agréger les réponses

Si vous avez des tâches complexes, mais que vous n'avez pas besoin de les effectuer dans un ordre spécifique, vous pouvez exécuter des requêtes parallèles et agréger les réponses du modèle.

Exemple

Par exemple, supposons que vous possédiez un magasin de disques et que vous souhaitiez utiliser un modèle pour vous aider à choisir les disques à stocker en fonction des tendances de streaming musical et des données de vente de votre magasin.

Tâche 1 : analyser les données

La première étape consiste à analyser les deux ensembles de données : les flux de données et les données de ventes. Vous pouvez exécuter les requêtes pour effectuer ces tâches en parallèle. Les requêtes qui effectuent ces tâches peuvent être semblables à ce qui suit, où *STORE_SALES_DATA* est un fichier contenant les données de vente et *STREAMING_DATA* est un fichier contenant les flux de données :

1a : analyser les données de vente

```
e:

yze the sales data to identify the number of sales of each record.
se organize the output into a JSON format with each record as the key, and sales as
```

```
e.

:: STORE_SALES_DATA
```

La sortie devrait contenir le nombre de ventes pour chaque disque, au format JSON.

1b : analyser les flux de données

```
e:

yze the streaming data to provide a the number of streams for each album.
se organize the output into a JSON format with each album as the key, and streams a
value.

:: STREAMING_DATA
```

La sortie devrait contenir le nombre de diffusions pour chaque album, au format JSON.

Tâche 2 : agréger les données

Vous pouvez maintenant agréger les données des deux ensembles de données pour vous aider à planifier vos décisions d'achat. Pour agréger les données, incluez la sortie des deux tâches en tant qu'entrée. Une requête qui effectue cette opération peut ressembler à ce qui suit, où *TASK_1A_RESPONSE* et *TASK_1B_RESPONSE* sont les réponses des tâches précédentes :

er des données sur les ventes et les flux de données

```
e:

mmend a stocklist of about 20 records based on the most sold and most streamed recc
nly three quarters of the stock list should be based on record sales, and the rest
aming.

:: TASK_1A_RESPONSE and TASK_1B_RESPONSE
```

La sortie devrait contenir une liste de suggestions de stock d'environ 20 disques, en fonction des ventes et des diffusions, avec une préférence pour les disques ayant un historique de ventes avéré plutôt que ceux qui sont plus populaires en streaming.

Étapes suivantes

- Découvrez des exemples de requêtes dans la galerie de requêtes (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/prompt-gallery?hl=fr).

Sauf indication contraire, le contenu de cette page est régi par une licence Creative Commons Attribution 4.0 (https://creativecommons.org/licenses/by/4.0/), et les échantillons de code sont régis par une licence Apache 2.0 (https://www.apache.org/licenses/LICENSE-2.0). Pour en savoir plus, consultez les Règles du site Google Developers (https://developers.google.com/site-policies?hl=fr). Java est une marque déposée d'Oracle et/ou de ses sociétés affiliées.

Dernière mise à jour le 2026/01/28 (UTC).